Maze-Solving Rover: Code

Yantrika: Robotics and Coding Club, RSA, IACS

Board: Arduino Uno (Nano works too, same pins). **Serial Monitor:** 9600 baud.
To drive the rover: battery switch off, upload over USB, unplug USB, battery switch on.

Part	Pin
Left motor direction	IN1 = D7, IN2 = D8
Right motor direction	IN3 = D9, IN4 = D10
Ultrasonic sensor	TRIG = D12, ECHO = D11
Servo	D3
ENA and ENB	Left on their 5V jumpers. Not wired to the Arduino.

How to copy: select a whole code box with the mouse, copy, and paste it into the Arduino IDE. There are no line numbers, so the code pastes cleanly.

Because ENA and ENB are on 5V jumpers, the motors always run at full speed and the code only uses the direction pins IN1 to IN4. Do not connect D5 or D6 to ENA or ENB while the jumpers are fitted.

Step 1: Motor test

Lift the rover so the wheels spin in the air.

```
// STEP 1: Motor test - each motion runs for 2 seconds
// Lift the rover so the wheels spin in the air!
// ENA and ENB are jumpered to 5V, so the motors always run at full speed.

const uint8_t IN1 = 7, IN2 = 8;     // left motor
const uint8_t IN3 = 9, IN4 = 10;    // right motor

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  delay(2000);                      // time to step back
}

void drive(bool lFwd, bool rFwd) {
  digitalWrite(IN1, lFwd); digitalWrite(IN2, !lFwd);
  digitalWrite(IN3, rFwd); digitalWrite(IN4, !rFwd);
}

void stopMotors() {
  digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
}

void loop() {
  drive(true, true);   delay(2000);   // forward
  stopMotors();        delay(1000);
  drive(false, false); delay(2000);   // backward
  stopMotors();        delay(1000);
  drive(false, true);  delay(2000);   // spin left
  stopMotors();        delay(1000);
  drive(true, false);  delay(2000);   // spin right
  stopMotors();        delay(1000);
}
```

Step 2: Ultrasonic sensor test

Open the Serial Monitor at 9600 baud. The battery switch can stay off.

```
// STEP 2: Distance sensor test

const uint8_t TRIG = 12, ECHO = 11;

void setup() {
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  Serial.begin(9600);
}

long readCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);   // timeout 30 ms (about 5 m)
  if (us == 0) return 400;                // no echo: nothing detected
  return us / 58;
}

void loop() {
  Serial.print("Distance: ");
  Serial.print(readCm());
  Serial.println(" cm");
  delay(200);
}
```

Step 3: Servo test

Sensor on the servo horn; 90 degrees must point straight ahead. If 160 looks right instead of left, swap LOOK_LEFT and LOOK_RIGHT in Step 4.

```
// STEP 3: Servo test - sweep left, centre, right, centre
#include <Servo.h>

const uint8_t SERVO_PIN = 3;
Servo head;

void setup() {
  head.attach(SERVO_PIN);
  head.write(90);                    // centre
  delay(1000);
}

void loop() {
  head.write(160); delay(1000);      // should look LEFT
  head.write(90);  delay(1000);      // centre: sensor points straight ahead
  head.write(20);  delay(1000);      // should look RIGHT
  head.write(90);  delay(1000);      // centre
}
```

Step 4: Maze-solving rover

Needs the Servo library (included with the Arduino IDE). The rover follows the left wall (left-hand rule): look, decide, move one step, repeat.

```
/*
  STEP 4: MAZE-SOLVING ROVER  (left-hand rule)

  Idea: keep one hand on the left wall and you will eventually
  find the way out of most mazes. At every step the rover:

     1. LOOKS   left, right and ahead
     2. DECIDES using the left-hand rule:
           left wall ended? -> turn left
           else ahead open? -> go straight
           else right open? -> turn right
           else             -> dead end, turn around
     3. MOVES   forward one short step, then repeats

  NOTE: the L298N ENA and ENB pins are jumpered to 5V, so the motors
  always run at full speed. Only the direction pins (IN1-IN4) are used.
*/

#include <Servo.h>

// ---- Pins ----
const uint8_t IN1 = 7, IN2 = 8;      // left motor direction
const uint8_t IN3 = 9, IN4 = 10;     // right motor direction
const uint8_t TRIG = 12, ECHO = 11;  // ultrasonic
const uint8_t SERVO_PIN = 3;

// ---- Tunable settings (change these to match your maze) ----
const int TURN_MS_90   = 300;   // time to turn about 90 degrees (calibrate!)
const int STEP_MS      = 400;   // how long to drive between looks (about one corridor width)

const int OPEN_CM      = 25;    // farther than this = the way is open
const int SAFE_CM      = 12;    // while driving, stop if something is this close
const int FINISH_CM    = 80;    // left, front AND right all farther than this = maze solved

// Servo angles. If the rover turns TOWARD a wall instead of away
// from it, swap LOOK_LEFT and LOOK_RIGHT.
const int LOOK_LEFT = 160, LOOK_CENTER = 90, LOOK_RIGHT = 20;

Servo head;

// Remembers whether there was a wall on the left at the previous step.
// Without this, an open area makes the rover turn left forever (circling).
bool wallWasOnLeft = false;

// ======================================================
//  SENSING
// ======================================================

// One distance reading in cm. 400 means "no echo = nothing in range".
long readCm() {
  digitalWrite(TRIG, LOW);  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
  long us = pulseIn(ECHO, HIGH, 30000);
  if (us == 0) return 400;
  return us / 58;
}

// Take 3 readings and return the middle one, so a single bad echo is ignored.
long readCmSmooth() {
  long a = readCm(); delay(10);
  long b = readCm(); delay(10);
  long c = readCm();
  return max(min(a, b), min(max(a, b), c));
}

// Turn the head to an angle, wait for it to arrive, then measure.
long lookAt(int angle) {
```

```
  head.write(angle);
  delay(500);
  return readCmSmooth();
}

bool isOpen(long cm) {
  return cm > OPEN_CM;
}

// ====================================================
//  MOTION
// ====================================================

// One motor: dir = +1 forward, -1 backward, 0 stop
void setMotor(uint8_t inA, uint8_t inB, int dir) {
  digitalWrite(inA, dir > 0);
  digitalWrite(inB, dir < 0);
}

void drive(int left, int right) {
  setMotor(IN1, IN2, left);
  setMotor(IN3, IN4, right);
}

void stopMotors() { drive(0, 0); }

void turnLeft() {
  drive(-1, 1);                 // left wheels back, right wheels forward
  delay(TURN_MS_90);
  stopMotors();
}

void turnRight() {
  drive(1, -1);
  delay(TURN_MS_90);
  stopMotors();
}

void turnAround() {
  drive(1, -1);
  delay(TURN_MS_90 * 2);
  stopMotors();
}

// Drive straight for STEP_MS, but stop early if a wall gets too close.
void driveOneStep() {
  unsigned long start = millis();
  drive(1, 1);
  while (millis() - start < STEP_MS) {
    if (readCmSmooth() < SAFE_CM) break;
  }
  stopMotors();
}

// ====================================================
//  SETUP
// ====================================================

void setup() {
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);
  pinMode(TRIG, OUTPUT); pinMode(ECHO, INPUT);

  head.attach(SERVO_PIN);
  head.write(LOOK_CENTER);
  Serial.begin(9600);
  delay(2000);                   // 2 s to put the rover in the maze
}

// ====================================================
//  MAIN LOOP:  look -> decide -> move
// ====================================================

void loop() {
```

```
  // 1. LOOK (the head finishes facing forward)
  long left  = lookAt(LOOK_LEFT);
  long right = lookAt(LOOK_RIGHT);
  long front = lookAt(LOOK_CENTER);

  Serial.print("Left: ");   Serial.print(left);
  Serial.print("  Front: "); Serial.print(front);
  Serial.print("  Right: "); Serial.println(right);

  // 2. FINISHED? Wide open on every side means we are out of the maze.
  if (left > FINISH_CM && front > FINISH_CM && right > FINISH_CM) {
    stopMotors();
    Serial.println("Maze solved!");
    while (true) { }             // stay here forever
  }

  // 3. DECIDE: the left-hand rule
  //    Turn left only if we were following a wall that has now ended.
  //    If there is no wall on the left yet, drive on until we find one.
  bool leftOpen = isOpen(left);

  if (leftOpen && wallWasOnLeft) {
    Serial.println("-> turn LEFT (the left wall has an opening)");
    turnLeft();
    wallWasOnLeft = true;        // the new corridor should have a left wall
  } else if (isOpen(front)) {
    Serial.println("-> straight");
    wallWasOnLeft = !leftOpen;   // remember if a left wall is there
  } else if (isOpen(right)) {
    Serial.println("-> turn RIGHT");
    turnRight();
    wallWasOnLeft = true;        // the wall that was ahead is now on our left
  } else {
    Serial.println("-> dead end, turn AROUND");
    turnAround();
    wallWasOnLeft = true;
  }

  // 4. MOVE
  driveOneStep();
}
```

Tuning and troubleshooting

• **TURN_MS_90:** test turnLeft() and turnRight() alone until each turn is close to 90 degrees. Calibrate this first, and again after charging the battery.

• **STEP_MS:** about one corridor width of travel between looks.

• **OPEN_CM:** a little more than the distance to the wall when the rover is centred in a corridor, and less than the distance to a real opening.

• **FINISH_CM:** larger than any gap inside the maze, or it may declare "solved" too early.

• **Turns toward a wall instead of away:** swap LOOK_LEFT and LOOK_RIGHT, or swap the motor wires on one side.

• **Circles in an open area:** it needs a wall on its left to follow. Place it beside a wall and it will track along it.